Badji Mokhtar Annaba University

Mini Project:

Building a Machine Learning-Based Classification Model to Differentiate between Normal and Malicious Network

Realized by : Zerdaoui Akram

Bouzenzen Loubna

Supervised by: Korba

Cyber-Security - Master GADM

Academic Year : 2022/2023.

Table of contents

I. Table of contents ……...

II. Figures table …….

III. Introduction ……..

1.1. Types of Cyber attacks……..

IV. Part 1 : Dos attack……..

2.1. Starting the work ……

1. Capturing the traffic using wireshark……...

V. Part 2 : Data analysis …….

3.1. Dataset making and preprocessing……..

1. Using cicflowmeter ……

2. Fixing the csv files ……

3.2. Writing the models code ……

1. Decision tree model …….

2. The MLP (Multilayer Perceptron) Model……..

VI. Conclusion

Figures Table

Figure 1 . Active attack 5

Figure 2. Dos attack 7

Figure 3. metasploit console while running a syn flood 8

Figure 4. wireshark capturing the syn packets sent from kali linux 9

Figure 5. performing the udp flood attack on the windows machine 10

Figure 6 . wireshark capturing the UDP packets sent from kali linux 10

Figure 7. Cicflowmeter interface 12

Figure 8. Dataset after removing the attributes 13

Figure 9 .importing the Libraries 14

Figure 10. Data importing 14

Figure 11. Labeling the data 15

Figure 12. Making the big dataset 15

Figure 13. Cleaning the dataset 15

Figure 14. Splitting our data 16

Figure 15. creating the model classifier 16

Figure 16. Finding the important features 16

Figure 17. Predicting 17

Figure 18. Calculating the accuracy 17

Figure 19. F1 and recall evaluation 17

Figure 20. Confusion Matrix..... 18

Figure 21. 5-fold cross validation 18

Figure 22. Importing the dataset 19

Figure 23. Cleaning the data 19

Figure 24. encoding the data to numerical..... 20

Figure 25. Concatinating the datasets 20

Figure 26. splitting the data 20

Figure 27. Applying the MLP model 21

Figure 28. MLP accuracy 21

Figure 29. MLP model accuracy 22

Figure 30. Confusion Matrix and heatmap 22

Figure 31. MLP Heatmap 23

Figure 32. important attributes using in the training..... 24

Figure 33. New MLP model accuracy 24

Figure 34. New MLP model accuarcy plot 24

Network security is a crucial concern in our technology-driven world, and identifying and mitigating malicious network traffic is a major challenge. Machine learning algorithms have shown promise in identifying cyber threats. In this essay, we will explore the development of a machine learning-based classification model to differentiate between normal and malicious network traffic, and investigate its effectiveness in improving network security.

But before we dive into machine learning and the prepossessing of the dateset let's take a look about what type of cyber attacks we are going to perform and how does it actually work in real life ?

1.1 Types of Cyber attacks

Cyber attacks are malicious activities aimed at compromising the security, confidentiality, or availability of computer systems, networks, or data. They can be divided into two categories**:** **passive** and **active attacks** ..

Passive attacks involve monitoring and gathering information, while **active attacks** involve disrupting, modifying, or destroying targeted systems or data.

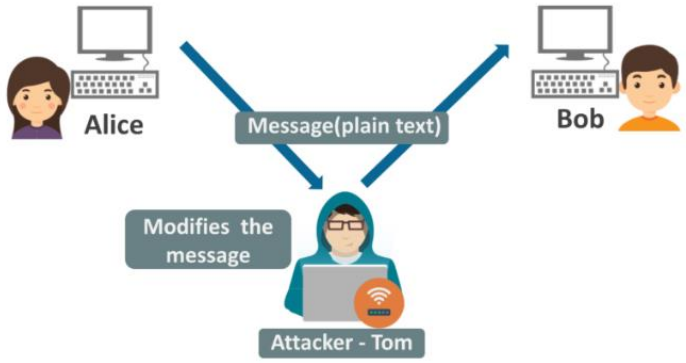


Figure 1 . Active attack

So in this project we will focus on one of the active attacks that is called the Denial of service (DoS Attack) , but what is exactly a Dos attack ?

1. The Denial of Service

Denial of Service (DoS) is a type of cyber attack in which a malicious actor aims to render a computer or another device unavailable to its users by interrupting the normal operation of the device.

DoS attacks typically work by flooding a targeted machine with requests until normal traffic is unable to be processed.

- As we said , it works by flooding a targeted machine , what do we exactly mean by flooding ?

In the context of computer networks and cyber attacks, flooding refers to a technique used to overwhelm a target device or network with a large amount of traffic or data packets , Dos attacks has several types of flooding like SYN flooding , UDP flooding , PING flooding and more …

In this project we will perform 3 of the common types which are :

- SYN flood
- UDP flood
- ICMP flood

1.1. SYN flood

In a SYN flood attack, the attacker sends a large number of SYN requests to a target server, but never completes the three-way handshake process. This ties up server resources and prevents legitimate users from accessing the server.

1.2. UDP flood

In a UDP flood attack, the attacker sends a large number of UDP packets to a target device or network, overwhelming it with traffic and making it unavailable to legitimate users.

1.3. ICMP flood

the attacker sends a large number of ping requests (ICMP Echo Requests) to a target device or network, overwhelming it with traffic and making it unavailable to legitimate users.

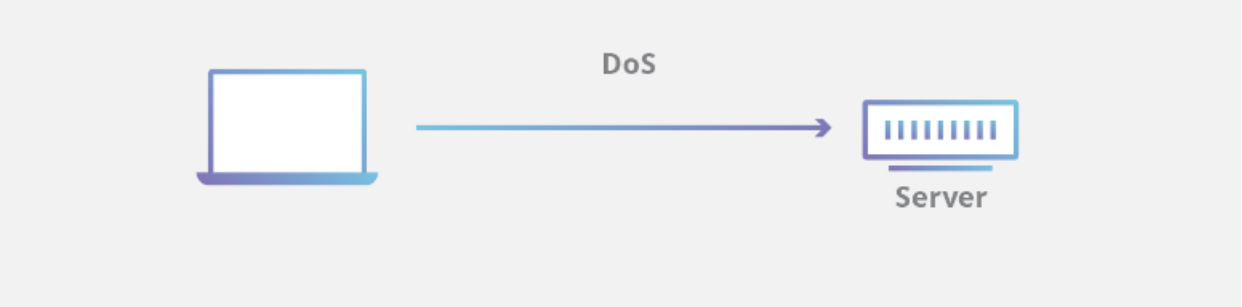


Figure 2. Dos attack

2.1 Starting the work

1. Capturing the traffic using wireshark

First of all let's introduce our machines , as the attacker machine we will have the Linux virtual machine with the kali Linux OS with the IP address "192.168.1.8" , the victim machine will be the windows physical machine with the IP address "192.168.1.3".

1.1. Capturing the SYN flood traffic

- We go to our kali Linux terminal, we go super user and type **msfconsole,** the **metasploit console** starts then we type the following lines to start the SYN flooding :

✓ $use auxiliary/dos/tcp/synflood

✓ $set RHOST adresse IP de la machine victim

✓ $set NUM 0

✓ $run

```
msf6 > use auxiliary/dos/tcp/synflood
msf6 auxiliary(dos/tcp/synflood) > set RHOST 192.168.1.3
RHOST => 192.168.1.3
msf6 auxiliary(dos/tcp/synflood) > set NUM 0
NUM => 0
msf6 auxiliary(dos/tcp/synflood) > run
[*] Running module against 192.168.1.3

[*] SYN flooding 192.168.1.3:80 ...
```

Figure 3. metasploit console while running a syn flood

- As we can see the flooding starts and if we check wireshark in the victim machine we will be able to see all the traffic that's being sent from the attacker:

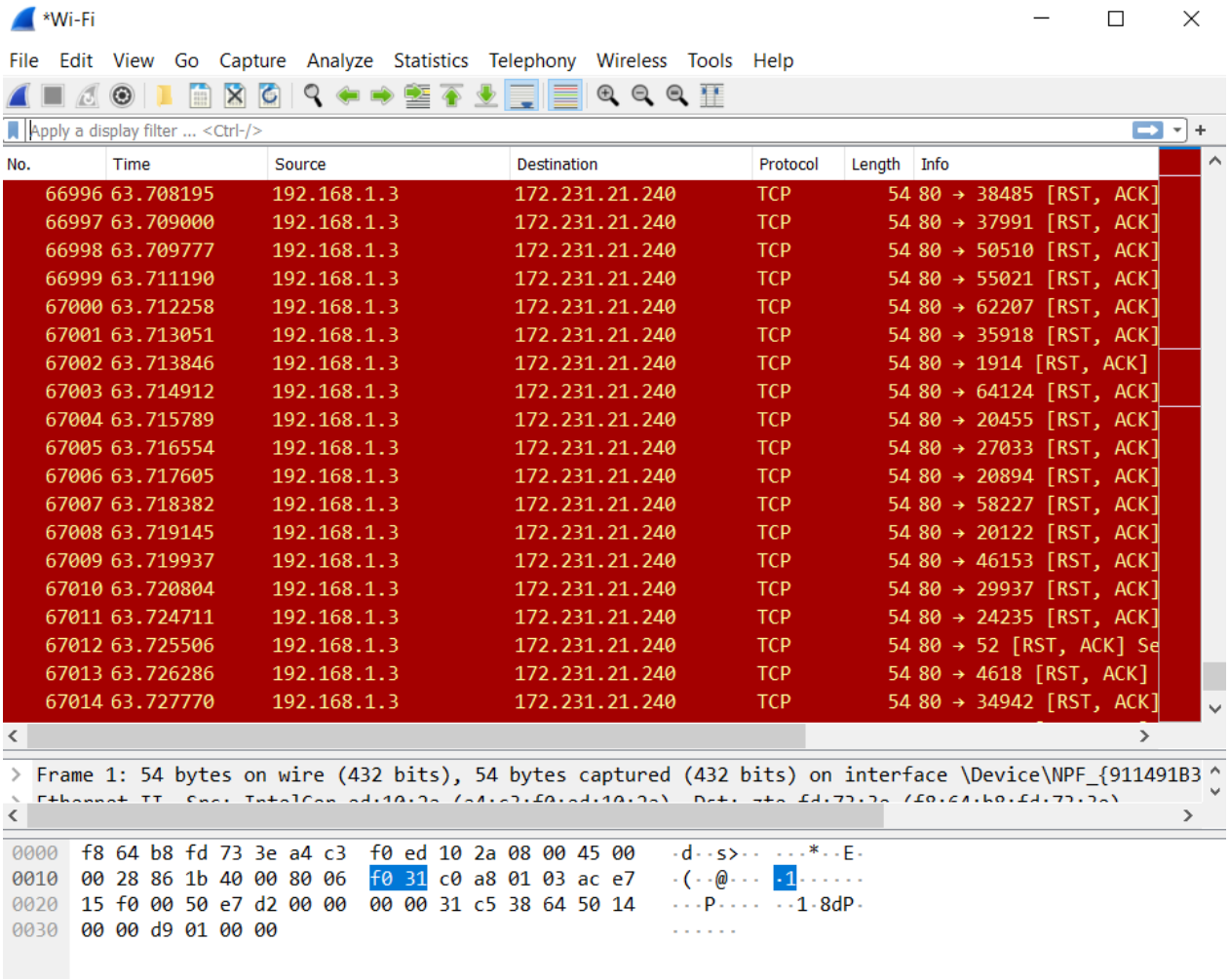


Figure 4. wireshark capturing the syn packets sent from kali linux

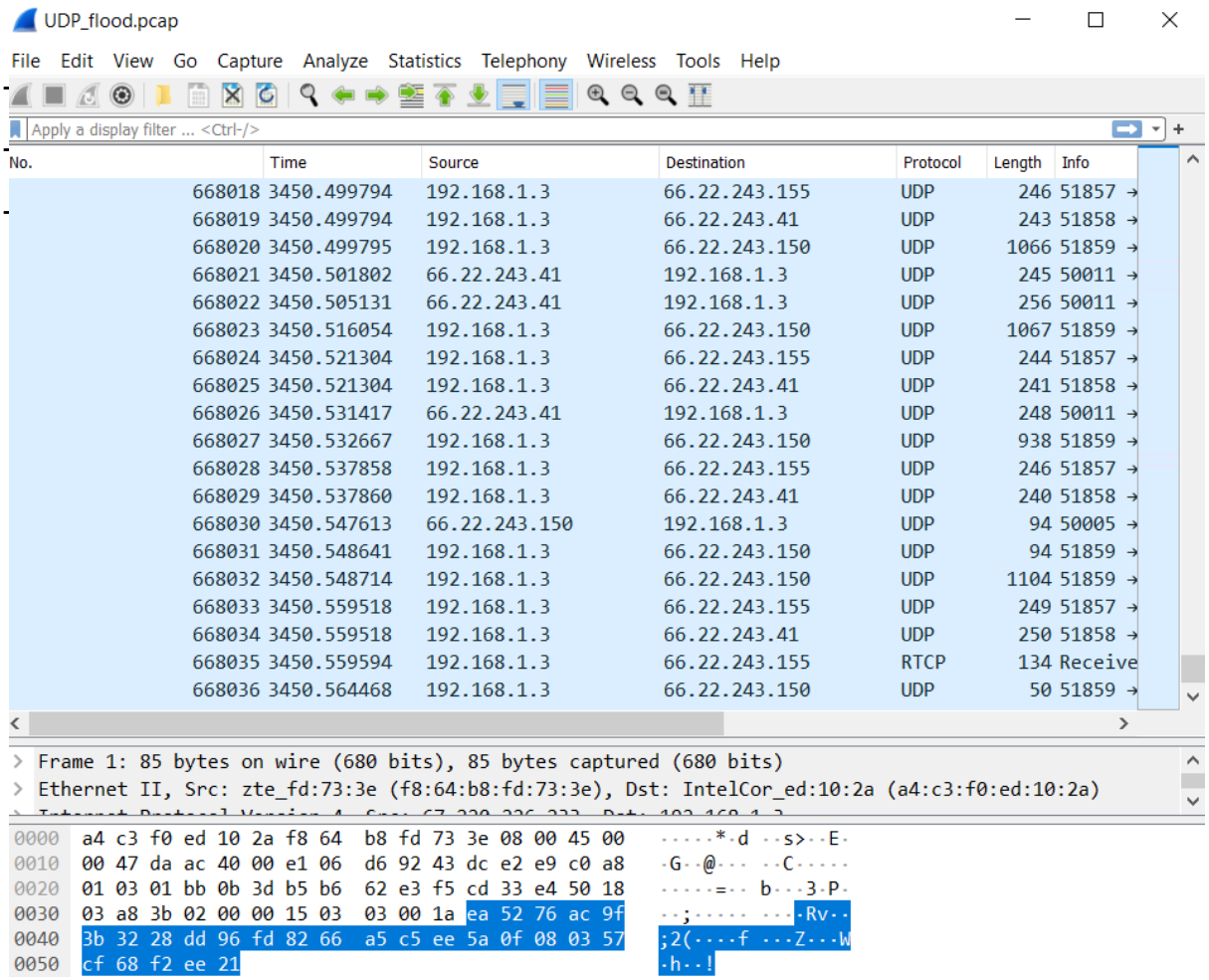
- We leave it like that for around 10 to 20 min so we can capture as much traffic as possible so that will help us later in the machine learning part .
- After the time passes we save the traffic as a pcap file **"SYN FLOOD.pcap"** ,stop the flooding from the metasploit console by typing CTRL+C and we go to perform the rest of the floodings.

1.2. Capturing the UDP flood traffic

- To perform this type of flood we go to our Linux terminal and type the following line :
  - ✓ $ sudo hping3 -2 192.168.1.3

```
┌──(kali㉿kali)-[~]
└─$ sudo hping3 --udp 192.168.1.3
[sudo] password for kali:
HPING 192.168.1.3 (eth0 192.168.1.3): udp mode set, 28 headers + 0 data bytes
ICMP Port Unreachable from ip=192.168.1.3 name=192.168.1.3
status=0 port=2147 seq=0
ICMP Port Unreachable from ip=192.168.1.3 name=192.168.1.3
status=0 port=2148 seq=1
ICMP Port Unreachable from ip=192.168.1.3 name=192.168.1.3
status=0 port=2149 seq=2
```

Figure 5. performing the udp flood attack on the windows machine



No.	Time	Source	Destination	Protocol	Length	Info
668018	3450.499794	192.168.1.3	66.22.243.155	UDP	246	51857 →
668019	3450.499794	192.168.1.3	66.22.243.41	UDP	243	51858 →
668020	3450.499795	192.168.1.3	66.22.243.150	UDP	1066	51859 →
668021	3450.501802	66.22.243.41	192.168.1.3	UDP	245	50011 →
668022	3450.505131	66.22.243.41	192.168.1.3	UDP	256	50011 →
668023	3450.516054	192.168.1.3	66.22.243.150	UDP	1067	51859 →
668024	3450.521304	192.168.1.3	66.22.243.155	UDP	244	51857 →
668025	3450.521304	192.168.1.3	66.22.243.41	UDP	241	51858 →
668026	3450.531417	66.22.243.41	192.168.1.3	UDP	248	50011 →
668027	3450.532667	192.168.1.3	66.22.243.150	UDP	938	51859 →
668028	3450.537858	192.168.1.3	66.22.243.155	UDP	246	51857 →
668029	3450.537860	192.168.1.3	66.22.243.41	UDP	240	51858 →
668030	3450.547613	66.22.243.150	192.168.1.3	UDP	94	50005 →
668031	3450.548641	192.168.1.3	66.22.243.150	UDP	94	51859 →
668032	3450.548714	192.168.1.3	66.22.243.150	UDP	1104	51859 →
668033	3450.559518	192.168.1.3	66.22.243.155	UDP	249	51857 →
668034	3450.559518	192.168.1.3	66.22.243.41	UDP	250	51858 →
668035	3450.559594	192.168.1.3	66.22.243.155	RTCP	134	Receive
668036	3450.564468	192.168.1.3	66.22.243.150	UDP	50	51859 →

Figure 6 . wireshark capturing the UDP packets sent from kali linux

- And now we just do the same thing as previously, we open wireshark and we can see the traffic that is being sent , we leave it for around 30 min and then we save the file as **"UDP Flood.pcap"**and stop the UDP flooding from the Linux terminal.

1.3. Capturing the ICMP flood traffic

- To perform this type of flood we go to our Linux terminal and type the following line :
  - ✓ $ sudo hping3 --icmp 192.168.1.3 or $ sudo hping3 -1 192.168.1.3
- We save the traffic also as a pcap file “ icmp.pcap” and stop the flooding from the terminal.

3.1 Datasets making and prepossessing

1. Using CICFlowmeter

CICFlowMeter is an open-source network traffic analysis tool that is used for monitoring and analyzing network traffic flows. It can be used to detect network anomalies, perform network performance analysis, and identify security threats.

We will be using cicflowmeter to transfer our pcap files to csv files so we can perform our model on them later …

So we do have our 4 pcap files , “NORMAL flood” “SYN flood” “UDP flood” and finally the “ICMP flood” file , we will use the cicflowmeter to convert all of the to csv files as it shows in the following screenshot :

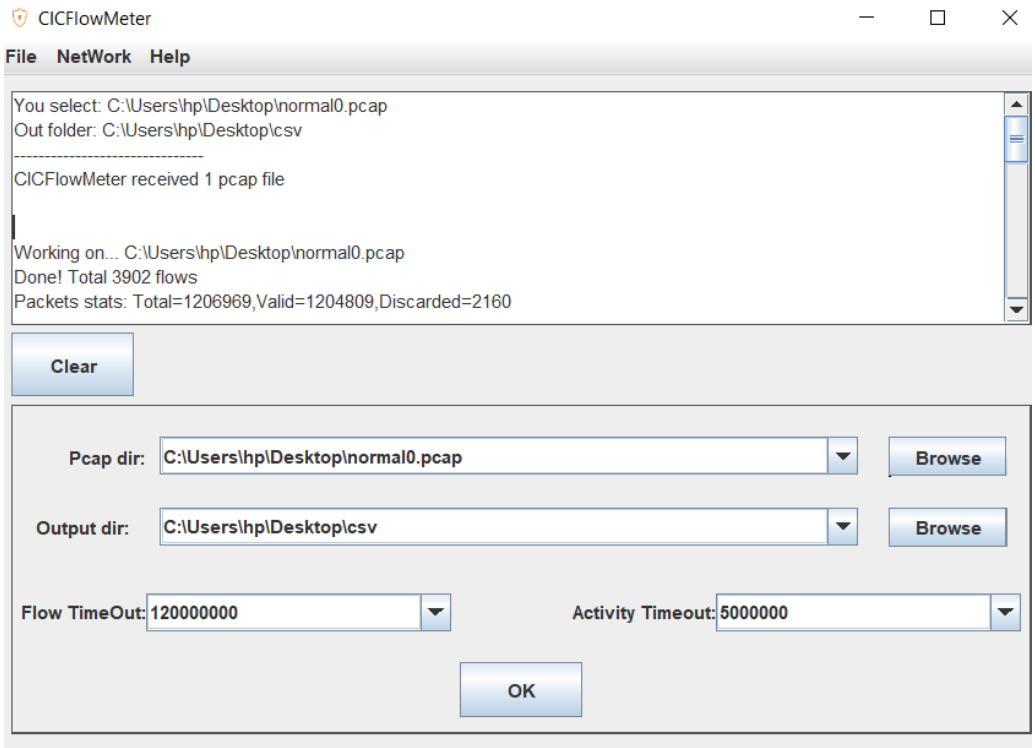


Figure 7. Cicflowmeter interface

- As we can see in the previous screenshot we did transfer our Normal flood pcap to a csv file and it gave us around 4000 valid flows , it did all the prepossessing for us and removed all the none valid packets .
- We do the same thing with the other pcap files .

2. Fixing the csv files

Now after we had our csv files ready , each one with 75 attributes , we have to remove 5 of these attributes as a last step to start our machine learning process .

- These 5 attributes are :
  - ✓ Source and destination IP
  - ✓ Source and destination port
  - ✓ Timestamps
- So after we remove them our datasets will be something like that :

	A	B	C	D	E	F	G	H	I	J	K
1	Protocol	Flow Dura	Tot Fwd Pk	Tot Bwd P	TotLen Fw	TotLen Bw	Fwd Pkt Le	Fwd Pkt Le	Fwd Pkt Le	Fwd Pkt Le	Bwd Pkt Le
2	6	15575	0	2	0	0	0	0	0	0	0
3	6	5270818	11	53	1052	51321	517	0	95.63636	198.1455	1150
4	6	0	1	1	0	31	0	0	0	0	31
5	6	0	1	1	0	31	0	0	0	0	31
6	6	0	1	1	0	31	0	0	0	0	31
7	6	0	1	1	0	31	0	0	0	0	31
8	6	233012	1	2	0	31	0	0	0	0	31
9	6	1281621	1	1	0	0	0	0	0	0	0

Figure 8. Dataset after removing the attributes

- ➢ Why did we remove these attributes ?

- Source and destination IP addresses, as well as source and destination ports, are not necessarily indicative of an attack, and most of the time they are not valuable in the detection process. Their real contribution to model formation is almost nonexistent. Most programming languages, such as Python, become confused when they encounter a number with three decimal points (IP addresses).

3.2 Writing the model code

- for the modeling part we have used 2 different models , first one is a decision tree model that gave us an over-fitting that we couldn't resolve and the second one is the MLP deep learning model that gave us a good and acceptable accuracy rate of around 70 % …

1. The Decision Tree Model

1- We start by uploading the necessary libraries that we used to prepare our data, shuffle it, create the model classifier and many more ..

Importing the necessary libraries

```
import numpy as np
import pandas as pd
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn import metrics
from sklearn.metrics import accuracy_score,confusion_matrix
from sklearn.metrics import classification_report
from sklearn.model_selection import cross_val_score
from sklearn.utils import shuffle
```

✓ 0.0s

Figure 9 .importing the Libraries

2- we import our data

Importing our data

```
UDP_flood= pd.read_csv('C:/Users/hp/Desktop/csv/UDP_flood.csv')
SYN_flood= pd.read_csv('C:/Users/hp/Desktop/csv/SYN_flood.csv')
ICMP_flood= pd.read_csv('C:/Users/hp/Desktop/csv/ICMP_flood.csv')
Normal_flood= pd.read_csv('C:/Users/hp/Desktop/csv/Normal_flood.csv')
```

Figure 10. Data importing

3- then we label all the flows

Labeling our data to normal and malicious

```
UDP_flood['Label'] = UDP_flood['Label'].replace({'No Label': 'DOS UDP '})
SYN_flood['Label'] = SYN_flood['Label'].replace({'No Label': 'DOS SYN '})
ICMP_flood['Label'] = ICMP_flood['Label'].replace({'No Label': 'DOS ICMP'})
Normal_flood['Label'] = Normal_flood['Label'].replace({'No Label': 'Normal'})
```

Figure 11. Labeling the data

4- we concatenate all the csv into one dateset

Concatinating our data in one dataset

```
dataset = pd.concat([SYN_flood, UDP_flood,ICMP_flood,Normal_flood])
```

Figure 12. Making the big dataset

5- we clean the dateset from the nan values and shuffle it since the labels are now ordered after the concatenation …

Removing the NAN values and shuffling the data

```
dataset = dataset.dropna()
dataset = shuffle(dataset)
```

Figure 13. Cleaning the dataset

6- we split our data into training and testing data

```
x= dataset[[ 'RST Flag Cnt','Pkt Len Max' ,'Flow IAT Min','Init Bwd Win Byts','Flow IAT Max','Bwd IAT
 'Bwd Pkts/s','Pkt Len Min','Fwd Seg Size Avg','Fwd IAT Min','Bwd Header Len','Fwd IAT Tot
'Bwd Seg Size Avg','Fwd Act Data Pkts','Pkt Len Var','Idle Min','Subflow Fwd Byts','PSH Flag Cnt','Fw
'Bwd IAT Max','Fwd IAT Std','Fwd IAT Mean','Bwd IAT Std','FIN Flag Cnt','Fwd Header Len','Active Max'

y=dataset['Label']
# split the dataset into training and test sets
X_train, X_test, y_train, y_test = train_test_split( x,y,test_size=0.3,random_state=42)
```

Figure 14. Splitting our data

7- we create the decision tree classifier and fit it to the training data

```
# create the classifier object
clf = DecisionTreeClassifier()

# fit the classifier to the training data
clf.fit(X_train, y_train)
```

Figure 15. creating the model classifier

8- we use the important features library to extract the most important attributes to correctly train our model and give better results

```
# get the feature importances
importances = clf.feature_importances_

# create a dataframe to store the feature importances
feature_importances = pd.DataFrame({'feature': X_train.columns, 'importance': importances})
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)

# sort the features by importance in descending order
feature_importances = feature_importances.sort_values('importance', ascending=False)

# print the feature importances
print(feature_importances)
```

Figure 16. Finding the important features

9- we predict on the test data

```
# make predictions on the test data
y_pred = clf.predict(X_test)
```

Figure 17. Predicting

10- we calculate the accuracy and here we can see the over-fitting since it gave us a huge accuracy of 98 % …

```
# calculate the accuracy of the classifier
accuracy = accuracy_score(y_test, y_pred)

# print the accuracy of the classifier
print("Accuracy:", accuracy)
```

Figure 18. Calculating the accuracy

11- we do an F1 score and a recall evaluation, result can show us that the over-fitting is happening on the syn flood labeled flows …

```
              precision    recall  f1-score   support

    DOS ICMP       0.94      0.86      0.90       749
    DOS SYN        1.00      1.00      1.00      7612
    DOS UDP        0.97      0.99      0.98      4687
      Normal       0.98      0.98      0.98      6084

    accuracy                           0.99     19132
   macro avg       0.97      0.96      0.97     19132
weighted avg       0.99      0.99      0.99     19132
```

Figure 19. F1 and recall evaluation

12- we do a confusion matrix evaluation

```
confusion_mat = confusion_matrix(y_test, y_pred)
print("Confusion matrix:")
print(confusion_mat)
```

✓ 0.3s

```
Confusion matrix:
[[2905    2   24   69]
 [   2 3858    0    3]
 [   7    0 3829   16]
 [  77    3  101 5952]]
```

Figure 20. Confusion Matrix

13 – and finally we do a 5 cross validation to our dateset

5-fold cross validation

```
# perform 5-fold cross-validation and calculate the mean accuracy
scores = cross_val_score(clf, x, y, cv=5)
mean_accuracy = scores.mean()

# print the mean accuracy
print("Mean accuracy:", mean_accuracy)
```

✓ 14.5s

```
Mean accuracy: 0.9865377163736616
```

Figure 21. 5-fold cross validation

2. The MLP (Multilayer Perceptron) Model

MLP is a type of artificial neural network that is commonly used for supervised learning tasks such as classification and regression. It is a type of deep learning model that can learn hierarchical representations for data through multiple layers of nonlinear transformations. MLPs are particularly effective when working with numerical values and have been used in a wide range of applications including image and speech recognition.

Why MLP ?

MLP models are well-suited for detecting flood attacks that involves numerical data because they are designed to work with numerical inputs and are particularly effective at handling high dimensional numerical data.

1. Importing the Datasets

Data Preperation

```
import pandas as pd

#Importing SYN_flood dataset
df1 = pd.read_csv("SYN_flood.csv")

#Importing ICMP_flood dataset
df2 = pd.read_csv("ICMP_flood.csv")

#Importing UDP_flood dataset

df3 = pd.read_csv("UDP_flood.csv")

#Importing Normal dataset

df4 = pd.read_csv("Normal_flood.csv")
```

Figure 22. Importing the dataset

2. Cleaning “infinity” and “nan” values

Cleaning "infinity" and "nan" values

```
[ ] import numpy as np
    #Creating a list of the datasets
    datasets = [df1, df2, df3, df4]
    #Replacing the infinity values by the maximum value in the dataset + 1
    for dataset in datasets:
        dataset = dataset.replace([np.inf, -np.inf], np.nan)
        df_num = dataset.select_dtypes(include=np.number)
        max_val = df_num.max(skipna=True).max()
        dataset.fillna(max_val + 1, inplace=True)
```

Figure 23. Cleaning the data

3. **Since MLP is effective with numerical values we encoded the output from string format to integer**

Coding outputs from string format to integer

```
#SYN_flood = 1, ICMP_flood = 2, UDP_flood = 3, Normal = 4
df1['Label'] = df1['Label'].replace('SYN_flood', 1)
df2['Label'] = df2['Label'].replace('ICMP_flood', 2)
df3['Label'] = df3['Label'].replace('UDP_flood', 3)
df4['Label'] = df4['Label'].replace('Normal', 4)
```

Figure 24. encoding the data to numerical

4. **We concatenate the 4 datasets and then we mix them**

Concatenate the 4 datasets into one dataset

```
[ ] my_dataset = pd.concat([df1, df2, df3, df4], axis=0).sample(frac=1)
    my_dataset=my_dataset.dropna()
    print(my_dataset.iloc[:10, -2:])
```

```
          Idle Min  Label
14799         0.0    4.0
11278  13500000.0    3.0
17139  51200000.0    1.0
13601  50900000.0    1.0
20421  55300000.0    1.0
4017   17000000.0    1.0
604           0.0    2.0
8996    9900455.0    1.0
9837          0.0    4.0
14395  97200000.0    1.0
```

Figure 25. Concatinating the datasets

5. **splitting data into input and output**

Splitting attributs into a list of inputs and an output

We used all the attributes

```
[ ] X = my_dataset.drop('Label', axis=1)
    y = my_dataset['Label']
```

```
[ ] #Importing necessary libraries from sklearn
    from sklearn.neural_network import MLPClassifier
    from sklearn.model_selection import train_test_split
    from sklearn.metrics import accuracy_score
```

Figure 26. splitting the data

6. MLP model implementation

Applying MLP Deep Learning Model

```
[ ] #Splitting the dataset into train and test
    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
[ ] #Calling MLPClassifier
    mlp = MLPClassifier(hidden_layer_sizes=(100,), max_iter=500, alpha=0.0001,
                        solver='adam', verbose=10, random_state=42, tol=0.0001)
```

```
[ ] #Training MLPClassifier
    mlp.fit(X_train, y_train)
```

```
Iteration 1, loss = 12.60060408
Iteration 2, loss = 11.21938640
Iteration 3, loss = 10.87113128
Iteration 4, loss = 10.49959185
Iteration 5, loss = 10.16115846
Iteration 6, loss = 10.10704923
Iteration 7, loss = 9.91033182
Iteration 8, loss = 9.95151988
Iteration 9, loss = 9.95823341
Iteration 10, loss = 9.69148979
```

Figure 27. Applying the MLP model

7. Model Accuracy and Accuracy Plot

Model Accuracy

```
[ ]
    acc = accuracy_score(y_test, y_pred)
    print('Accuracy:', acc)
```

```
Accuracy: 0.6998827667057445
```

Figure 28. MLP accuracy

The Plot :

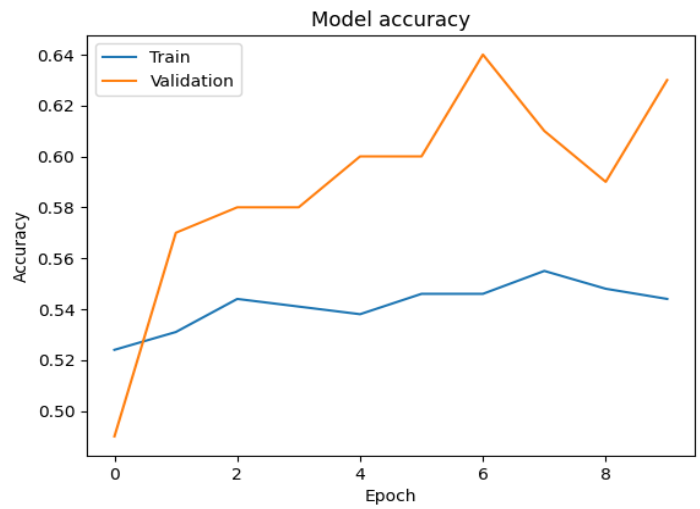


Figure 29. MLP model accuracy

8. Confusion Matrix and heat map:

Model Confusion Matrix

```
[ ] from sklearn.metrics import confusion_matrix

    cnf_matrix = confusion_matrix(y_test, y_pred)
    cnf_matrix

    array([[4305,    1,  674,   10],
           [  24,  328,  119,   71],
           [  70,  110, 2189,  814],
           [  49,  192, 1706, 2133]])

    import seaborn as sns

    # plot confusion matrix as a heatmap
    sns.heatmap(cnf_matrix, annot=True, cmap='Greens')
```

Figure 30. Confusion Matrix and heatmap

The Heatmap:

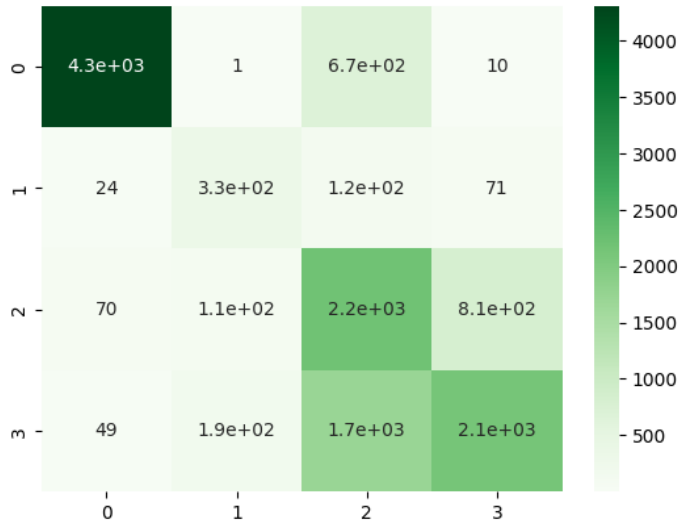


Figure 31. MLP Heatmap

Confusion Matrix Interpretation

- The model correctly classified most instances of SYN_flood(1), with only 1 instance misclassified as ICMP_flood(3) and 10 instances classifies as normal.
- The model had some difficulty classifying instances of ICMP_flood(2), with 71 instances as normal.
- The model had the most difficulty classifying instances of UDP_flood , with 70 instances misclassified as SYN_flood, 110 instances misclassified as ICMP_flood, and 814 instances misclassified as normal.
- The model preformed well in classifying instances of normal, with only 49 instances misclassified as SYN_flood, 192 instances misclassified as ICMP_flood, and 1706 instances misclassified as UDP_flood.

Now with repeating same process except the list of inputs X has less attributes:

We got higher accuracy of 76%:

Splitting attributs into a list of inputs and an output

We used 43/78

```
[ ] X = my_dataset[['RST Flag Cnt', 'Pkt Len Max' , 'Flow IAT Min' ,  'Init Bwd Win Byts', 'Flow IAT Max', 'Bwd IAT Min', 'Subflow Bwd Byts', 'Fwd Pkts/s', 'Flow IAT Mean',
                    'Bwd IAT Tot', 'Bwd Pkts/s', 'Pkt Len Min', 'Fwd Seg Size Avg', 'Fwd IAT Min', 'Bwd Header Len', 'Fwd IAT Tot','Flow IAT Std', 'Pkt Size Avg', 'Pkt Len Std',
                    'Bwd Seg, Size Avg','Fwd Act Data Pkts', 'Pkt Len Var', 'Idle Min',  'Subflow Fwd Byts', 'PSH Flag Cnt', 'Fwd IAT Max', 'Pkt Len Mean',  'Idle Std',
                    'Active Std', 'Bwd IAT Mean', 'Bwd IAT Max', 'Fwd IAT Std', 'Fwd IAT Mean', 'Bwd IAT Std', 'FIN Flag Cnt',  'Fwd Header Len','Active Max', 'Active Min',
                    'Active Mean', 'Idle Mean', 'ACK Flag Cnt']]
    y = my_dataset['Label']
```

Figure 32. important attributes using in the training

```
[ ] acc = accuracy_score(y_test, y_pred)
    print('Accuracy:', acc)

    Accuracy: 0.7624071903087143
```

Figure 33. New MLP model accuracy

Accuracy Plot:

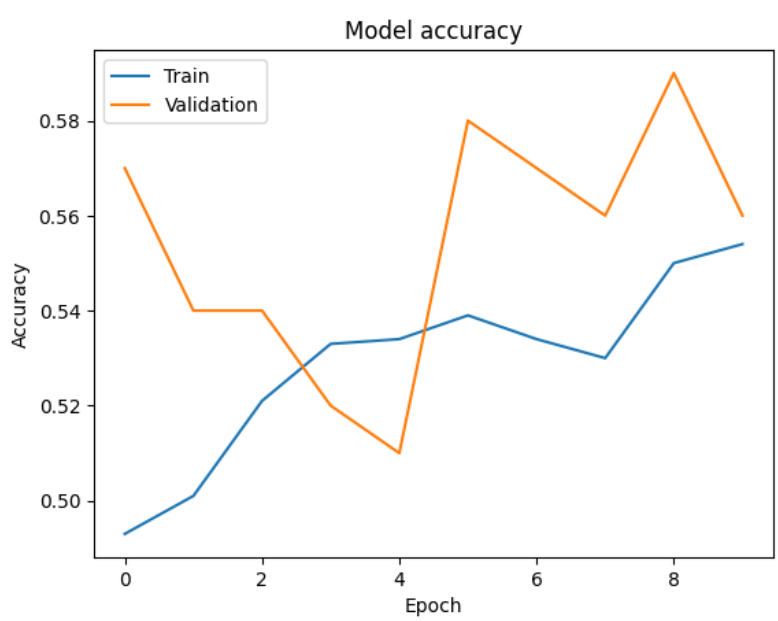


Figure 34. New MLP model accuarcy plot

Conclusion

After the completion of this Project by using the machine learning model decision tree to detect the denial of service attacks, we have observed that the model gave us results that arent that accurate and a huge overfitting appeared in almost all of the flows detection as we saw in the F1 and recall evaluation . However, the best model used is the deep learning model MLP, which gave us an accuracy rate of 0.69 and 0.74 when we trained it on the important attributes ....